AI-BASED SMART INTERVIEW SYSTEM

INT 500 – INTERNSHIP 4

PROJECT REPORT

Submitted by

Mr. NARESH KUMAR B – E0322008

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Data Analytics)

Sri Ramachandra Faculty of Engineering and Technology
Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai - 600116

MAY 2026

AI-BASED SMART INTERVIEW SYSTEM

INT 500 – INTERNSHIP 4

PROJECT REPORT

Submitted by

Mr. NARESH KUMAR B – E0322008

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Data Analytics)

Sri Ramachandra Faculty of Engineering and Technology
Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai - 600116

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI-BASED SMART INTERVIEW SYSTEM" is the bonafide record of work done by "Mr. NARESH KUMAR B – E0322008" who carried out the internship work under my supervision.

Signature of the Mentor **Signature of the HOD**

Dr. Purushothaman. R
Assistant Professor,
Department of Artificial Intelligence and Data Analytics
Sri Ramachandra Faculty of Engineering and Technology,
SRIHER, Porur, Chennai – 600116

Evaluation Date:

Examiner 1:

Examiner 2:

ACKNOWLEDGEMENT

I take this opportunity to express my gratitude to Prof. T. Ragunathan, Dean and Prof. A. Saravanan, Vice Dean, Sri Ramachandra Faculty of Engineering and Technology, SRIHER, for providing all the facilities to complete this internship project successfully.

I express my sincere gratitude to Dr. A. Sathya, Programme Coordinator, Department of Artificial Intelligence and Data Analytics, SRET, for providing the required support and academic guidance for carrying out this study.

I also extend my sincere gratitude to Dr. R. Purushothaman, our Internship Coordinator, whose efforts in facilitating my internship were vital in this learning experience.

I would like to express my deepest appreciation to my supervisor and project guide for extending help and encouragement throughout the project work. I am also thankful to Wittmann Battenfeld India Pvt Ltd for giving me an opportunity to work on a company-focused smart interview system.

I am grateful to all faculty members, my parents, and my friends for their constant support and encouragement during the completion of this project.

TABLE OF CONTENTS

ABSTRACT

The AI-Based Smart Interview System is a web-based intelligent interview assessment platform developed to automate first-level candidate screening. Traditional interview screening requires manual effort, repeated evaluation, and more time when many candidates apply for similar roles. This project addresses the problem by providing a structured interview system that verifies candidates, assigns role-based questions, evaluates answers using Natural Language Processing, monitors candidate face presence using computer vision, stores interview data, and generates a professional report.

The backend of the system is implemented using Python Flask, while the frontend is developed using HTML, CSS, and JavaScript. Candidate verification is performed through OTP-based authentication. After verification, candidates can select roles such as Manual Testing, Automation Testing, AI Tech Support, and Software Development. The system uses exact answer matching for objective questions and keyword matching, TF-IDF cosine similarity, and communication scoring for descriptive answers.

OpenCV is used to perform basic camera monitoring such as face presence detection, centered-face validation, and multiple-face alerts. PostgreSQL is used as the database for the live question bank, candidate details, assigned interview papers, answers, scores, shortlist status, and report paths. The final output is a PDF interview report that supports admin review and shortlist decisions.

The project demonstrates how AI, NLP, computer vision, and database-backed web applications can support a consistent, low-cost, and scalable candidate screening process.

KEYWORDS: Artificial Intelligence, Smart Interview System, NLP, TF-IDF, OpenCV, Flask, PostgreSQL, Candidate Screening.

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

Interview assessment is an important process for selecting candidates based on technical knowledge, reasoning ability, communication skill, and professional behaviour. In a manual interview process, evaluators ask questions, listen to candidate responses, prepare notes, compare performance, and finally prepare a decision. This becomes difficult when many candidates have to be screened within a short time.

The AI-Based Smart Interview System is developed as an academic and internship project to support first-level candidate screening. The system is inspired by company-focused recruitment needs and uses a structured online interview workflow. It allows candidates to register, verify OTP, choose a role, attend an interview, and receive AI-based evaluation. The administrator can review generated reports and shortlist decisions.

1.1 Background of the Study

Recruitment and technical screening are increasingly supported by digital systems. Many organizations conduct online assessments before direct HR or technical interviews. However, simple online forms do not evaluate descriptive answers, communication quality, or basic interview monitoring. Therefore, an intelligent system that combines a web application, NLP-based scoring, computer vision monitoring, and automated report generation is useful for improving consistency and reducing manual effort.

1.2 Problem Statement

Manual first-level interview screening requires more time, depends heavily on evaluator availability, and may produce inconsistent feedback when many candidates are assessed. There is a need for a smart system that can conduct role-based interviews, evaluate candidate answers using measurable scoring logic, monitor basic candidate presence, store interview data, and generate structured reports for admin review.

1.3 Objectives of the Study

- To develop a Flask-based smart interview platform for candidate screening.
- To provide OTP verification and role-based interview flow.

- To evaluate answers using exact matching, keyword matching, TF-IDF cosine similarity, and communication scoring.
- To monitor basic face presence and multiple-face conditions using OpenCV.
- To store candidate details, question bank, answers, scores, and reports in PostgreSQL.
- To generate automated PDF reports for shortlist decision support.

1.4 Scope and Limitations

1.4.1 Scope

The scope of the project includes candidate registration, OTP verification, role selection, role-wise question assignment, AI-based answer evaluation, camera monitoring, report generation, and admin report viewing. The system is suitable for academic demonstration and first-level screening support.

1.4.2 Limitations

The system provides basic AI-based support and does not replace final human interview judgment. Face monitoring is limited to camera visibility and multiple-face alerts. The accuracy of descriptive scoring depends on the quality of ideal answers, keywords, and question-bank preparation. Advanced emotion analysis and deep semantic models are considered future enhancements.

1.5 Significance of the Study

The project is significant because it demonstrates a practical use of AI and data analytics in recruitment screening. It combines NLP, OpenCV, Flask, PostgreSQL, and PDF reporting into one working application. The system reduces repeated manual work and provides structured feedback for candidates and administrators.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction to the Literature

The literature review focuses on automated interview assessment, online video interview analysis, NLP-based answer scoring, machine learning in recruitment, computer vision face detection, and fairness in AI-based hiring systems. These studies help to understand how verbal content, non-verbal signals, and structured scoring can support interview evaluation.

2.2 Related Work

YEAR	AUTHORS	TECHNIQUE USED	KEY CONTRIBUTION	LIMITATION
2015	I. Naim et al.	Multimodal ML	Predicted interview performance using verbal and nonverbal cues.	Requires large interview datasets.
2016	L. Nguyen and D. Gatica-Perez	Video Analytics	Analyzed online video resumes for hirability impressions.	Focuses more on video resume context.
2017	L. Chen et al.	Speech + Vision	Used speech, facial, and language features for interview judgment.	High computational complexity.
2018	S. Muralidhar et al.	NLP	Showed importance of spoken content in candidate assessment.	Limited visual monitoring.
2011	F. Pedregosa et al.	Scikit-learn	Provided TF-IDF and similarity computation tools.	Library only, not a complete interview system.
2000	G. Bradski	OpenCV	Supported real-time image processing and face detection.	Detection quality depends on camera and lighting.
2019	H. Suen et al.	HR Analytics Review	Reviewed AI usage in personnel selection.	Highlights governance and fairness concerns.

2.2.1 Comparison of Existing System

System Type	Features	Limitations	Proposed Improvement
Manual Interview	Human judgment and direct interaction.	Time-consuming and inconsistent for large candidate groups.	Automated first-level screening and report generation.
Online Test Platform	Objective question scoring.	Limited descriptive answer evaluation and monitoring.	NLP scoring and camera presence checking.
Video Interview Platform	Records candidate responses.	Often needs manual review and paid services.	Local Flask-based system with low-cost AI scoring.
AI Recruitment Tools	Advanced analytics.	May require large datasets and paid APIs.	Academic local system using TF-IDF, OpenCV, and

			PostgreSQL.

2.3 Research Gaps

- Many existing systems focus only on objective test scores and do not evaluate descriptive answers.
- Some AI interview systems depend on paid APIs or large private datasets.
- Candidate monitoring is often either missing or too complex for small academic projects.
- There is a need for a simple integrated system that combines candidate workflow, NLP scoring, OpenCV monitoring, database storage, and automated reports.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Workflow Diagram

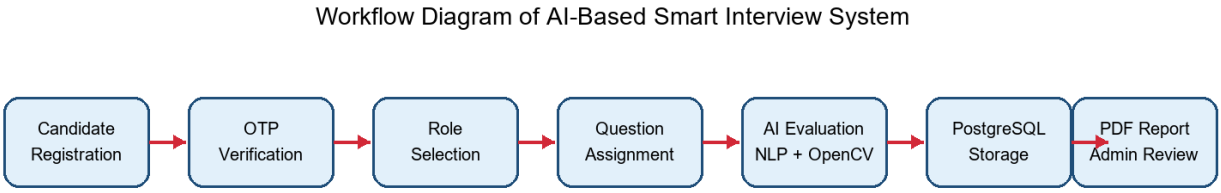

Figure 1: Workflow Diagram

3.2 Proposed System/Methodology

The proposed methodology follows a structured pipeline beginning with candidate registration and ending with report generation. Each module is implemented separately and connected through Flask routes and APIs. Candidate information and interview results are stored in the database, while scoring logic and camera monitoring are handled by separate utility modules.

3.2.1 Candidate Registration

The candidate enters name, email, phone number, and resume details through the web interface.

3.2.2 OTP Verification

A six-digit OTP is generated and verified before the candidate can proceed to the interview.

3.2.3 Role Selection

The candidate selects a role such as Manual Testing, Automation Testing, AI Tech Support, or Software Development.

3.2.4 Question Assignment

The PostgreSQL question bank provides role-wise aptitude and programming questions. The exact assigned paper is stored for consistency.

3.2.5 Answer Evaluation

Objective answers are evaluated using exact matching. Descriptive answers are evaluated using keyword matching, TF-IDF semantic similarity, and communication score.

3.2.6 Face Analysis

The webcam frame is sent to the backend API and OpenCV detects face presence, centered face status, and multiple-face alerts.

3.2.7 Report Generation

The final score, feedback, confidence index, and shortlist decision are added to a PDF report.

3.2.8 Deployment

The application runs as a Flask web application and can be accessed locally or through a LAN address with HTTPS support.

3.3 Module Description

Module	Description
Authentication Module	Handles candidate registration and OTP verification.
Question Bank Module	Stores and selects role-based aptitude and programming questions.
Scoring Module	Computes exact answer, keyword, TF-IDF, communication, and total score.
Computer Vision Module	Performs webcam-based face presence and multiple-face detection.
Report Module	Generates candidate-wise PDF interview reports.
Admin Module	Provides database and report viewing support for admin review.

3.4 Summary

The methodology integrates user interaction, AI evaluation, database operations, and reporting into a single workflow. This makes the system suitable for demonstrating AI-assisted first-level interview screening.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Introduction

System implementation converts the proposed methodology into a working Flask application. The implementation includes frontend pages, backend routes, NLP scoring utilities, OpenCV frame analysis, PostgreSQL database functions, and PDF report generation.

4.2 Hardware and Software Specifications

4.2.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel i3 or above
RAM	4 GB or above
Storage	500 MB free space for application and reports
Camera	Webcam for face monitoring
Network	Localhost or LAN access for demo

4.2.2 Software Requirements

Software	Purpose
Python	Backend programming language
Flask	Web application framework
PostgreSQL	Database for question bank and interview records
OpenCV	Face detection and camera monitoring
Scikit-learn	TF-IDF vectorization and cosine similarity
ReportLab	PDF report generation
HTML, CSS, JavaScript	Frontend interface and camera/speech support

4.3 Tools / Libraries Used

The major tools and libraries used in this project are Python, Flask, psycopg, PostgreSQL, OpenCV, scikit-learn, ReportLab, HTML, CSS, JavaScript, and browser speech recognition support. These tools were selected because they are suitable for local development and do not require paid AI APIs for the core functionality.

4.4 Implementation Details

4.4.1 Backend Implementation

The backend is implemented in app.py using Flask. It contains routes for home page, OTP verification, role selection, interview page, admin database view, admin reports, question API, frame analysis API, submit API, result page, report download, and resume download. Utility files are used for scoring, database operations, question-bank selection, and report creation.

4.4.2 Frontend Implementation

The frontend is implemented using HTML templates, CSS styling, and JavaScript. The interview page displays questions, captures answers, handles camera frames, updates face status, supports browser speech input, and submits the interview to the backend API.

4.5 Dataset Collection and Descriptions

The dataset for this project is the role-wise question bank prepared for the smart interview system. It contains aptitude and programming questions for roles such as Manual Testing, Automation Testing, AI Tech Support, and Software Development. Each question includes section, topic, difficulty, question text, options, correct answer, keywords, marks, and active status.

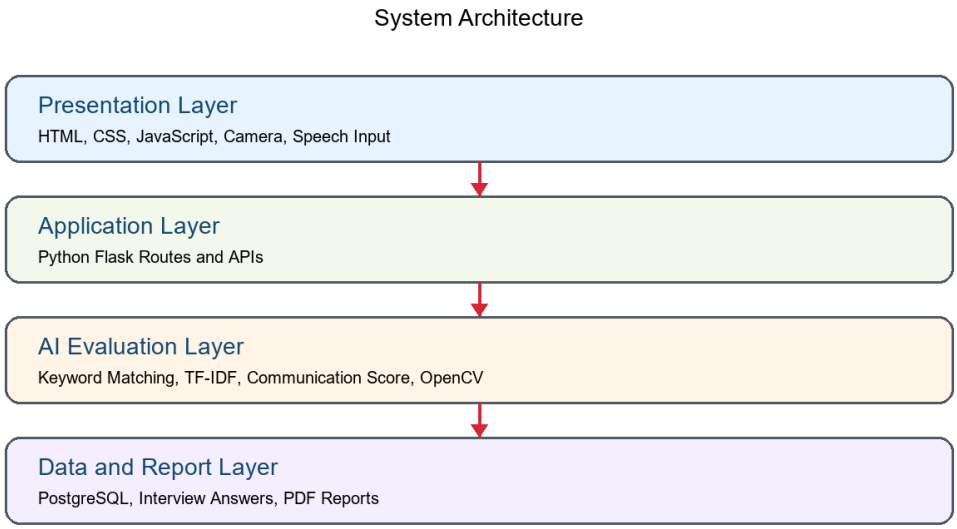

Figure 2: System Architecture

4.6 Summary

The implementation stage completed the major functional modules required for second and final review demonstration. The system currently supports candidate flow, AI scoring, camera monitoring, database storage, and automated report generation.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Experimental Results

The system was tested using sample candidate entries and role-based interview flows. Candidate registration, OTP verification, role selection, question loading, answer submission, AI scoring, face monitoring, report generation, and admin report viewing were verified successfully.

Feature Tested	Result
Candidate Registration	Working
OTP Verification	Working
Role Selection	Working
Question Loading	Working from PostgreSQL question bank
NLP Answer Scoring	Working using exact, keyword, TF-IDF, and communication score
Face Monitoring	Working using OpenCV face detection
PDF Report Generation	Working
Admin Report View	Working

5.2 Performance Comparison / Benchmarks

Evaluation Method	Purpose	Expected Output
Exact Answer Matching	Aptitude and programming objective answers	Correct / Incorrect score
Keyword Matching	Technical keyword coverage	Matched and missing keywords
TF-IDF Cosine Similarity	Semantic similarity between candidate answer and ideal answer	Similarity score
Communication Score	Answer length and vocabulary variety	Communication quality score
OpenCV Face Detection	Candidate presence monitoring	Face status and alert level

5.3 Analysis of Results

The results show that the system can complete the full assessment cycle without manual calculation. Objective questions are scored quickly using exact matching, while descriptive answers receive a combined score based on keyword relevance, semantic similarity, and communication quality. The camera module adds a basic confidence and monitoring layer to the interview process.

5.4 Discussion of Key Findings

- The integrated workflow reduces repeated manual effort in first-level screening.
- Role-wise question selection makes the interview more relevant to the candidate profile.

- The database-backed question bank makes the system easier to update and maintain.
- Automated reports provide structured information for admin review and shortlist decisions.

5.5 Limitations of Results

The current results are based on rule-based and classical NLP techniques. The system does not perform deep emotion analysis or advanced transformer-based semantic evaluation. Camera monitoring is limited by lighting, camera quality, and browser permissions. Human evaluation is still required for final hiring decisions.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Summary of the Research

This project developed an AI-Based Smart Interview System for automating first-level candidate screening. The system combines Flask, PostgreSQL, NLP scoring, OpenCV monitoring, and PDF report generation to provide a complete interview assessment workflow.

6.2 Conclusion Drawn from the Results

The project concludes that a smart interview system can support consistent and structured candidate screening. It helps reduce manual scoring effort, improves report preparation, and provides a repeatable process for role-based interviews. The system is suitable as an academic demonstration of AI, NLP, computer vision, and data analytics in recruitment.

6.3 Contributions of the Study

- Designed and implemented a complete web-based interview workflow.
- Integrated NLP scoring using exact answer matching, keyword matching, TF-IDF, and communication score.
- Added OpenCV-based camera monitoring for face presence and multiple-face alerts.
- Created a PostgreSQL-backed editable question bank and interview record system.
- Generated automated PDF reports for admin review.

6.4 Future Work

- Add HR/admin login with role-based access control.
- Improve semantic scoring using BERT or sentence-transformer models.
- Add dashboard analytics for candidate performance comparison.
- Expand the question bank with more company-specific and role-specific questions.
- Add video/audio recording support with stronger consent and privacy controls.
- Deploy the application on a secure cloud server for real-time use.

APPENDICES

Appendix 1 – Code

```
def score_answer(answer, question):
    kw = keyword_score(answer, question.get("keywords", []))
    sem = semantic_score(answer, question.get("ideal_answer", ""))
    comm = communication_score(answer)
    total = (kw * 0.35) + (sem * 0.45) + (comm * 0.20)
    return {"total_score": round(total * 100, 2),
            "matched_keywords": matched_keywords,
            "missing_keywords": missing_keywords}
```

Figure 3: Code Snapshot - NLP Scoring

```
def select_questions_for_role(role_slug, excluded_ids=None):
    for section, count in SECTION_COUNTS.items():
        pool = conn.execute("""
            SELECT * FROM question_bank
            WHERE role_slug = %s AND section = %s AND active = TRUE
        """, (role_slug, section)).fetchall()
        pool.sort(key=assignment_sort_key)
        selected.extend(pool[:count])
```

Figure 4: Code Snapshot - Question Bank

Appendix 2 – Screenshots

Screenshots of the candidate registration page, OTP page, role selection page, interview page, result page, admin database page, and generated report can be added here after the final demo screenshots are captured.

REFERENCES

[1] I. Naim et al., “Automated Prediction and Analysis of Job Interview Performance,” 2015.

[2] L. Nguyen and D. Gatica-Perez, “Hirability in the Wild: Analysis of Online Video Resumes,” 2016.

[3] L. Chen et al., “Automated Video Interview Judgment on Large Corpus,” 2017.

[4] S. Muralidhar et al., “Words Worth: Verbal Content and Hirability Impressions,” 2018.

[5] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” 2011.

[6] K. Sparck Jones, “A Statistical Interpretation of Term Specificity,” 1972.

[7] G. Bradski, “The OpenCV Library,” 2000.

[8] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers,” 2019.

[9] H. Suen et al., “Artificial Intelligence in Personnel Selection,” 2019.

[10] N. Guenole et al., “Algorithmic Recruitment and Selection Systems,” 2022.

[11] Python Software Foundation, Python Documentation.

[12] Flask Documentation, Pallets Projects.

[13] PostgreSQL Global Development Group, PostgreSQL Documentation.

[14] ReportLab User Guide for PDF Generation.